OpenAI Agent Implementation Tasks

Overview

This document provides a detailed, step-by-step breakdown of implementing the OpenAI Agent SDK integration into CosmoAudit. It expands on the IMPLEMENTATION TASKS from the requirements document (`docs/OpenAiSDK implementation.md`) and incorporates the architectural design from `docs/CosmoAudit_Architecture_Design.md` for seamless integration with the existing React frontend and Node.js backend.

The implementation adds a new Python-based microservice (Agent Service) that orchestrates AI-powered smart contract security audits using the OpenAI Agent SDK, while maintaining production-grade security, scalability, and testability.

Prerequisites

- CosmoAudit codebase with React frontend and Node.js backend
- Python 3.9+ environment
- OpenAI API access and API key
- Docker for containerization
- Git for version control
- Slither and Solidity compiler (solc) installed for local development

Sequential Implementation Tasks

1. Branch and Project Setup

Objective: Create a dedicated branch and establish the agent service directory structure.

Steps:

1. Create and checkout feature branch: `git checkout -b feat/openai-agent-sdk`
2. Create `agent_service/` directory in the project root
3. Create subdirectories: `agent_service/tools/` , `agent_service/prompts/` , `tests/` , `tests/fixtures/`

4. Initialize Python packages: Create `__init__.py` files in `agent_service/`, `agent_service/tools/`, `agent_service/prompts/`, and `tests/`
5. Update root `.gitignore` to exclude artifacts, logs, and temporary files
6. Create initial `requirements.txt` with core dependencies (fastapi, uvicorn, pydantic, openai)

Code Changes:

- New directory structure and empty `__init__.py` files
- Updated `.gitignore` with patterns for `artifacts/`, `*.log`, `__pycache__/`
- Basic `requirements.txt` with `fastapi==0.104.1`, `uvicorn==0.24.0`, `pydantic==2.5.0`, `openai==1.3.0`

Testing: Verify directory structure and Python imports work.

2. Core Infrastructure

Objective: Implement foundational components for data models, configuration management, and prompt templates.

Steps:

2.1 Implement `agent_service/schemas.py`

- Define Pydantic models for API requests/responses:
  - `AuditRequest`: repo_url, branch, contract_paths, run_explainer
  - `AuditResult`: run_id, report_url, summary (findings count, status)
  - `Finding`: name, severity, function, line, extra
  - `Explanation`: finding_id, risk_label, explanation, remediation_snippet, unit_test, confidence
- Add input validation for URLs and file paths

2.2 Implement `agent_service/config.py`

- Load environment variables with defaults:
  - `OPENAI_API_KEY` (required)
  - `VECTOR_DB_URL` (optional, placeholder)
  - `ARTIFACTS_DIR` (default: `./artifacts`)
  - `LOG_LEVEL` (default: INFO)
- Implement utility functions for secret redaction in logs
- Add configuration validation on startup

2.3 Implement `agent_service/prompts/prompts.py`

- Define system prompt template for AI explanations (based on requirements)

- Include structured input/output schemas
- Add prompt engineering safeguards against jailbreaks

Code Changes:

- `schemas.py` : Pydantic BaseModel classes with field validators
- `config.py` : Environment loading with `os.getenv()` and validation
- `prompts.py` : String templates with placeholders for dynamic content

Testing: Unit tests for schema validation and config loading.

3. Tool Wrappers Implementation

Objective: Implement pure function wrappers for all audit tools as specified in requirements.

Steps:

3.1 Implement `agent_service/tools/git_tool.py`

- Function signature: `git_tool(repo_url: str, branch: str, dest_dir: str) -> dict`
- Use `subprocess` to execute `git clone -b {branch} {repo_url} {dest_dir}`
- Parse git output for commit SHA
- Return `{"path": dest_dir, "commit": sha, "status": "ok"}` or `{"error": message}`

3.2 Implement `agent_service/tools/compile_tool.py`

- Function signature: `compile_tool(repo_path: str, contract_paths: List[str]) -> dict`
- Support both `solc` and `forge build` compilation
- Parse compiler output into structured format:
  `{"compiled": [{"file": path, "bytecode": hex, "abi": dict}], "errors": []}`
- Handle compilation failures gracefully

3.3 Implement `agent_service/tools/slither_tool.py`

- Function signature: `slither_tool(contract_paths: List[str], repo_dir: str) -> dict`
- Execute `slither --json {contract_paths}` in repo_dir
- Parse JSON output into findings list: `{"findings": [...], "raw": original_json}`
- Map severity levels and extract relevant metadata

3.4 Implement `agent_service/tools/search_tool.py`

- Function signature: `search_tool(query: str, top_k: int = 5) -> List[dict]`
- Placeholder implementation connecting to vector DB (mock for initial version)

- Return list of `{"id": str, "score": float, "snippet": str, "metadata": dict}`

3.5 Implement `agent_service/tools/explain_tool.py`

- Function signature: `explain_tool(findings: dict, code_snippets: List[str]) -> dict`
- Integrate OpenAI Agent SDK for structured explanations
- Use prompt templates from `prompts.py`
- Return `{"explanations": [...]}`
- Save prompts/responses to artifacts for auditability

3.6 Implement `agent_service/tools/report_tool.py`

- Function signature: `report_tool(payload: dict) -> dict`
- Save payload as JSON to `./artifacts/{run_id}/report.json`
- Generate local file URL
- Return `{"report_url": url, "id": run_id}`

Code Changes:

- Individual tool files with pure functions
- Use `subprocess` for external commands with proper error handling
- Add logging for tool execution and results
- Ensure all tools return machine-readable JSON

Testing: Unit tests mocking subprocess calls and external APIs.

4. Agent Core Implementation

Objective: Build the orchestration logic using OpenAI Agent SDK.

Steps:

4.1 Implement `agent_service/agent_core.py`

- Class `AgentRunner` with method `run_audit(req: AuditRequest) -> AuditResult`
- Generate unique `run_id` (UUID)
- Create artifacts directory `./artifacts/{run_id}/`

4.2 Implement sequential workflow:

- Call `git_tool` to clone repository
- Call `compile_tool` to compile contracts
- Call `slither_tool` to analyze contracts

- For each finding: call `search_tool` for RAG, then `explain_tool` for AI explanation
- Call `report_tool` to persist results

4.3 Integrate OpenAI Agent SDK:

- Import and configure agent with tool registration
- Use abstraction layer for SDK flexibility
- Handle agent planning and tool execution
- Persist all intermediate outputs (prompts, responses, tool outputs) to artifacts

Code Changes:

- `agent_core.py` with `AgentRunner` class
- OpenAI SDK integration with tool registration examples
- Artifact persistence logic for each step

Testing: Integration tests with mocked tools and OpenAI API.

5. API Layer Implementation

Objective: Create FastAPI application exposing audit endpoints.

Steps:
5.1 Implement `agent_service/app.py`

- FastAPI app with `POST /api/v1/audit` endpoint
- Request body validation using `AuditRequest` schema
- Inject `AgentRunner` dependency
- Return `AuditResult` with HTTP 200 on success
- Add error responses for validation failures

5.2 Add middleware and utilities:

- CORS middleware for frontend integration
- Request logging middleware
- Basic rate limiting (in-memory for initial version)

Code Changes:

- `app.py` with FastAPI routes and dependency injection
- Middleware setup for CORS and logging

Testing: API integration tests with test client.

6. Containerization

Objective: Dockerize the agent service for consistent deployment.

Steps:

6.1 Create `Dockerfile`

- Multi-stage build: builder stage for dependencies, runtime stage for execution
- Install system dependencies: git, python3, solc, slither
- Copy source code and install Python packages
- Expose port 8000, set CMD to `uvicorn app:app --host 0.0.0.0 --port 8000`

6.2 Update `requirements.txt`

- Add all dependencies: `pydantic`, `openai-agents` (if separate), `pytest`, etc.
- Pin versions for reproducibility

Code Changes:

- `Dockerfile` with minimal Alpine-based image
- Complete `requirements.txt` with all packages

Testing: Build Docker image and verify container runs.

7. Testing Implementation

Objective: Add comprehensive tests for reliability and CI/CD.

Steps:

7.1 Unit Tests (`tests/test_tools.py`)

- Test each tool wrapper with mocked subprocess and network calls
- Test error handling, edge cases, and input validation
- Use `pytest` with fixtures

7.2 Integration Tests (`tests/test_api_integration.py`)

- Test FastAPI endpoints with `TestClient`
- Create fixture repo in `tests/fixtures/vulnerable_repo/` with sample vulnerable Solidity contract
- Mock OpenAI API responses (do not call real API in CI)
- Smoke test full audit workflow

7.3 GitHub Actions Workflow (`.github/workflows/test-agent.yml`)

- Trigger on push/PR to `feat/openai-agent-sdk` branch
- Build Docker image, run unit tests, run integration tests
- Cache dependencies for faster builds

Code Changes:

- `tests/test_tools.py` with mocked tests
- `tests/test_api_integration.py` with fixture-based tests
- GitHub Actions YAML workflow
- Sample vulnerable contract in `tests/fixtures/`

Testing: All tests pass locally and in CI.

8. Documentation

Objective: Create comprehensive documentation for the agent service.

Steps:
8.1 Create `agent_service/README.md`

- Local development setup: install dependencies, run locally
- Environment variables documentation
- API endpoint documentation with examples
- Testing instructions (unit, integration, smoke test)
- Deployment guide

Code Changes:

- Detailed `README.md` with code examples and curl commands

9. Frontend Integration

Objective: Add audit functionality to the React frontend.

Steps:
9.1 Create audit request form component (`src/components/AuditRequestForm.tsx`)

- Input fields: repository URL, branch, contract paths (multi-select)
- Form validation and submission to agent service API

9.2 Update assessment results page (`src/pages/AssessmentResults.tsx`)

- Add "Request AI Audit" button after assessment completion

- Integrate audit request form

9.3 Create audit results display component (`src/components/AuditResults.tsx`)

- Display findings with severity badges
- Show AI explanations, remediation snippets, unit tests
- Handle loading states and errors

9.4 Update API service (`src/services/api.ts`)

- Add functions for audit requests and results fetching
- Handle async processing with polling or websockets

Code Changes:

- New React components with TypeScript
- API service functions for audit endpoints
- State management for audit status and results
- UI updates to existing pages

Testing: Frontend integration tests and manual UI testing.

10. Backend Integration

Objective: Integrate audit functionality with the Node.js backend.

Steps:
10.1 Extend PostgreSQL schema
- Add `audits` table: id, user_id, run_id, status, created_at, completed_at
- Link to existing `leads` and `assessments` tables

10.2 Add backend routes (`backend/routes/audit.js`)
- `POST /api/audit` to proxy requests to agent service
- Authentication middleware to ensure user is logged in
- Store audit metadata in database

10.3 Update existing routes
- Add audit status to assessment results endpoint

Code Changes:

- Database migration scripts
- New backend routes with proxy logic

- Update existing routes to include audit data

Testing: Backend API tests and database integration tests.

11. Deployment Setup

Objective: Configure production deployment on Railway.

Steps:
11.1 Update Railway configuration
- Add agent service as separate service in Railway dashboard
- Configure environment variables: OPENAI_API_KEY, etc.
- Set up networking between services

11.2 Load balancing and scaling
- Configure Railway for horizontal scaling
- Add health checks for agent service

11.3 Update deployment scripts
- Modify `RAILWAY_DEPLOYMENT.md` to include agent service
- Add agent service to `start-all.bat` or equivalent

Code Changes:

- Railway configuration files
- Updated deployment documentation
- Environment setup scripts

Testing: Deploy to staging and verify integration.

12. Security and Monitoring

Objective: Implement production security measures and observability.

Steps:
12.1 Add authentication and authorization
- API key authentication for agent service endpoints
- Rate limiting per IP/user (use Redis if available)
- Input sanitization for repository URLs and file paths

12.2 Implement logging and monitoring
- Structured JSON logging with correlation IDs

- Metrics collection (response times, error rates)
- Basic alerting for failures

12.3 Security hardening
- Secret redaction in logs and artifacts
- HTTPS enforcement
- Dependency vulnerability scanning

Code Changes:

- Authentication middleware
- Logging configuration
- Security utilities

Testing: Security audit and penetration testing.

13. Testing and Validation

Objective: Validate end-to-end functionality and performance.

Steps:
13.1 End-to-end testing
- Full audit workflow from frontend to backend
- Performance testing with load simulation
- Validate against acceptance criteria from requirements

13.2 User acceptance testing
- Beta testing with select users
- Collect feedback and iterate

13.3 Documentation updates
- Update main project README with audit feature
- Add troubleshooting guide

Code Changes:

- Additional test scripts for E2E
- Performance benchmark scripts
- Updated documentation

Acceptance Criteria

- [ ] `POST /api/v1/audit` returns 200 with valid JSON for fixture repo (with mocked LLM)
- [ ] Unit tests pass in CI for all tool wrappers and core logic
- [ ] Integration smoke test runs successfully with fixture repo
- [ ] Frontend displays audit request form and results correctly
- [ ] Backend proxies audit requests and stores metadata
- [ ] Agent service containerizes and deploys successfully
- [ ] All secrets are redacted in logs and artifacts
- [ ] Documentation is complete and accurate
- [ ] Performance meets requirements (<30s for typical audit)

Timeline and Milestones

- **Week 1**: Core implementation (tools, agent core, API, containerization)
- **Week 2**: Testing, documentation, and initial integration
- **Week 3**: Frontend and backend integration, deployment setup
- **Week 4**: Security, monitoring, final testing, and production deployment

Risk Mitigation

- Start with mocked OpenAI responses for development
- Implement comprehensive error handling and fallbacks
- Use feature flags for gradual rollout
- Monitor resource usage to prevent cost overruns

This implementation plan ensures a production-ready OpenAI Agent SDK integration that seamlessly extends CosmoAudit's capabilities while maintaining security, scalability, and user experience.